

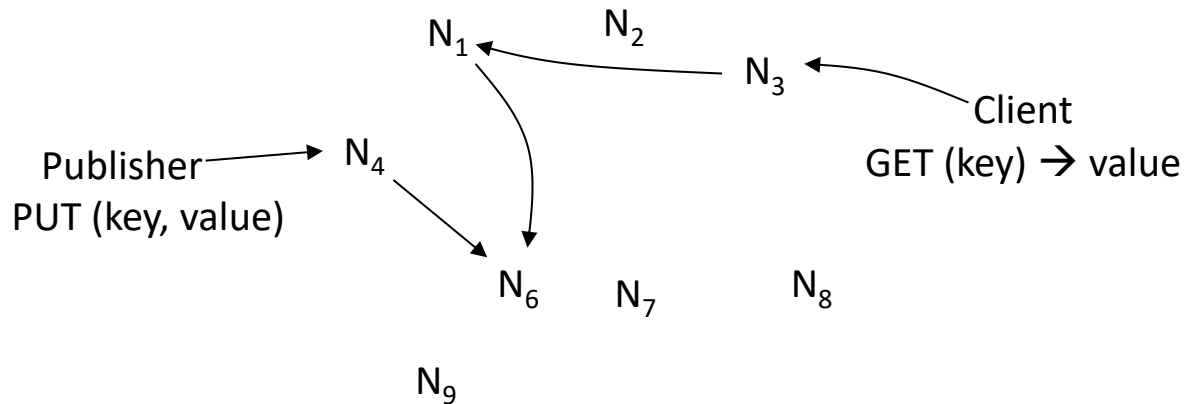
Consistent Hashing and Distributed Hash Tables

ECE 454: Distributed Computing

Instructor: Dr. Tahsin Reza

tahsin.reza@uwaterloo.ca

Distributed Hash Table (DHT)



- N_# is a node in the distributed system
- The hash table is distributed among nodes, a node stores multiple keys
- Any node can receive PUT/GET request from the external world
- PUT/GET requests are routed to the node that has the key

- Desired properties:

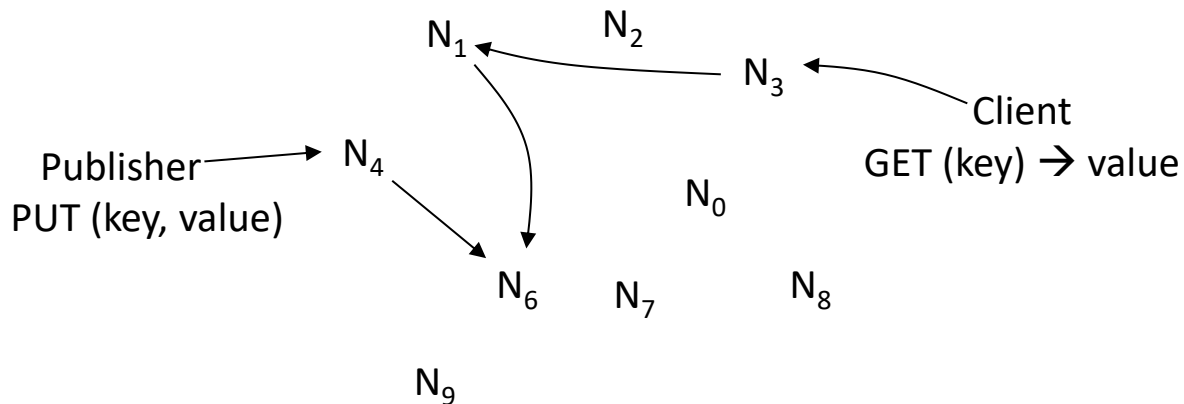
- Scalable: new nodes (servers) can be added to meet increasing user demands; the key space is partitioned and distributed among nodes, i.e., a node is responsible for storing a subset of the keys
- Accommodates node departure: nodes may fail, or leave the network
- Available: supports both nodes joining and departing without the system going offline
- Decentralized: no central coordinator, no single point of failure
- Fault tolerant, load balanced

- Observations:

- #unique keys to lookup >> #nodes
- Request routing can be simplified by encoding network routing information in hash keys

Naïve (and inefficient) solution

- Naming scheme: node ID space $[0, 1, 3, \dots, n)$
- Which node an object should be placed on: $\text{node_ID} = \text{hash}(\text{object_name}) \% \# \text{nodes}$
- One obvious limitation of this naming scheme is load imbalance
 - May lead to heavily skewed distribution of keys among nodes
- Not flexible to new nodes joining and leaving the network
 - Can you think of a reason why?



Consistent Hashing

- Key desired features of an efficient DHT:
 - A scalable distributed key-value store or hash table. Hash buckets are distributed over multiple nodes.
 - Update and lookup protocols that do not require users to have a current or even consistent view of the network. Nodes should have only partial knowledge of other nodes' keys.
 - Balanced: No hash bucket has disproportionate number of objects.
 - Smoothness: Addition/removal of a bucket (i.e., a node) does not cause data movement between all other existing buckets (data movement is limited to only a small number of buckets, e.g., immediate buckets).
- The original paper that presents the Consistent Hashing algorithm:

David Karger, Eric Lehman, Tom Leighton, Rina Panigrahy, Matthew Levine, and Daniel Lewin. 1997. Consistent hashing and random trees: distributed caching protocols for relieving hot spots on the World Wide Web. In Proceedings of the twenty-ninth annual ACM symposium on Theory of computing (STOC '97). Association for Computing Machinery, New York, NY, USA, 654–663.

Consistent Hashing

Real-world applications

- Content Delivery Network (CDN)
 - Akamai
 - Amazon CloudFront
 - Cloudflare
- Amazon Dynamo key-value store
- Apache Cassandra NoSQL database
- Peer-to-peer lookup service
- Distributed Hash Tables
 - Chord
 - Pastry
 - Tapestry
 - CAN
 - Kademlia

Content Delivery Network (CDN)

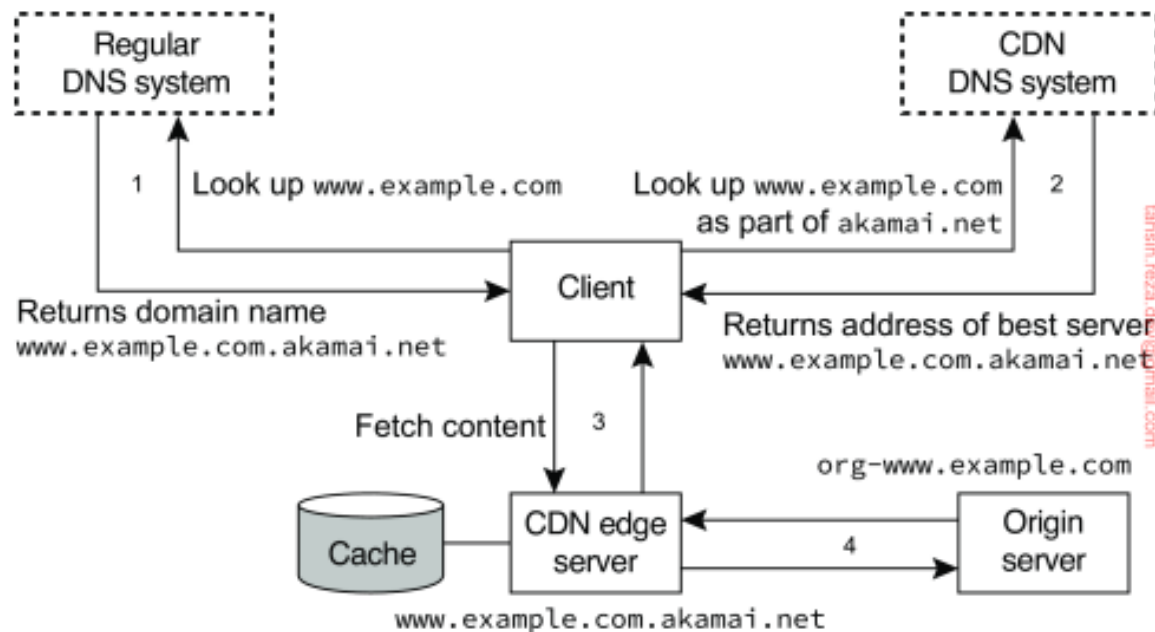


Figure 3.25: A simplified version of the working of the Akamai CDN.

Overlay Network

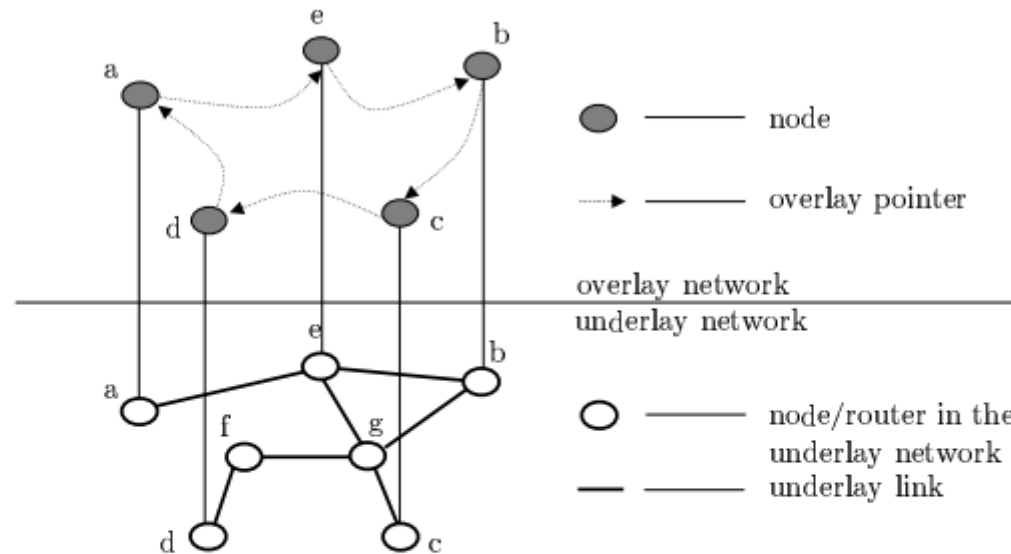
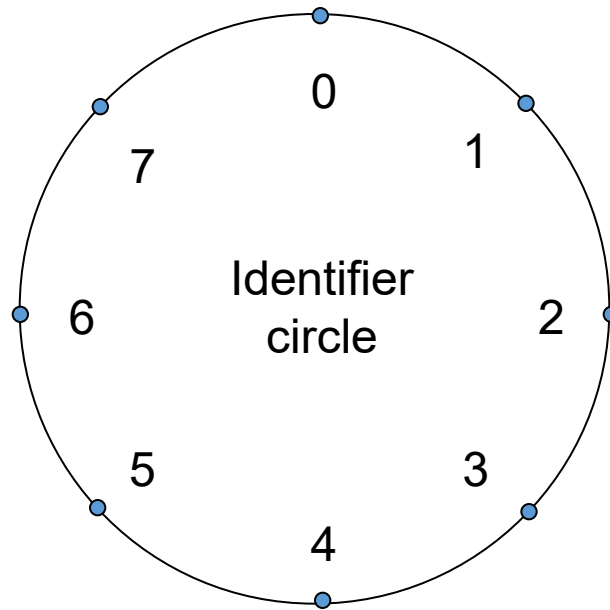


Figure 1.2: An overlay network and the underlay network on top of which the overlay network is built. Messages between the nodes in the overlay network logically follow the ring topology of the overlay network, but physically pass through the links and routers that form the underlay network.

A. Ghodsi, "Distributed k-ary System: Algorithms for Distributed Hash Tables," PhD dissertation, KTH, Stockholm, 2006.

Consistent Hashing

- An m -bit identifier space, total 2^m identifiers
- Identifiers are arranged on an “identifier circle”
- An m -bit key of an object can be generated using a hash function that produces a m -bit value and the key can be mapped to an identifier on the circle



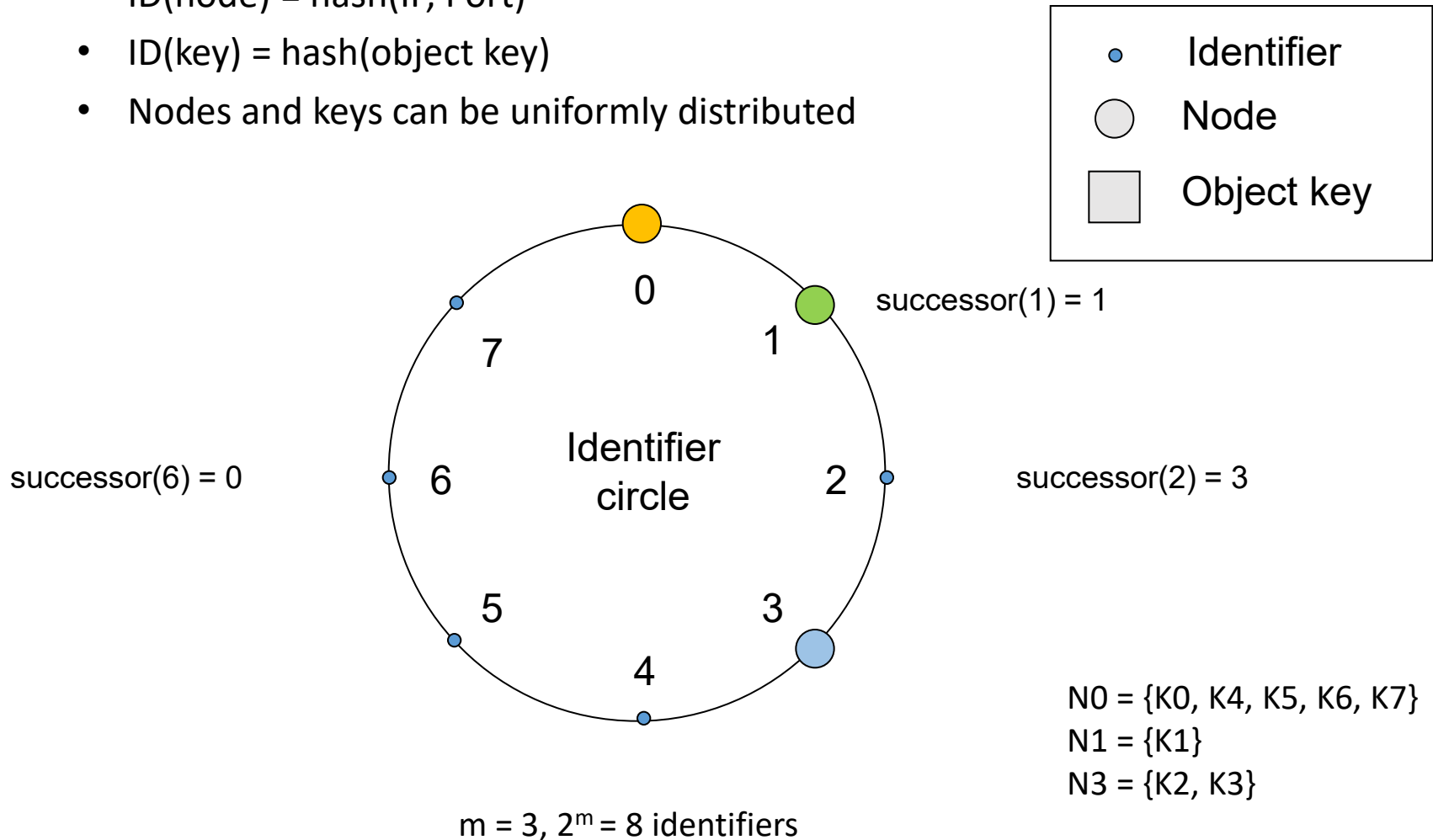
$m = 3, 2^m = 8$ identifiers

Example:

- SHA-1 hash function takes an input and produces a 160-bit (20-byte) hash value.
- When $m=160$, 2^{160} unique keys can be placed on the identifier circle.
- **Do we need 2^m physical nodes?**

Consistent Hashing

- Both nodes and object keys exist in the same m -bit ID space
- $ID(\text{node}) = \text{hash}(\text{IP, Port})$
- $ID(\text{key}) = \text{hash}(\text{object key})$
- Nodes and keys can be uniformly distributed



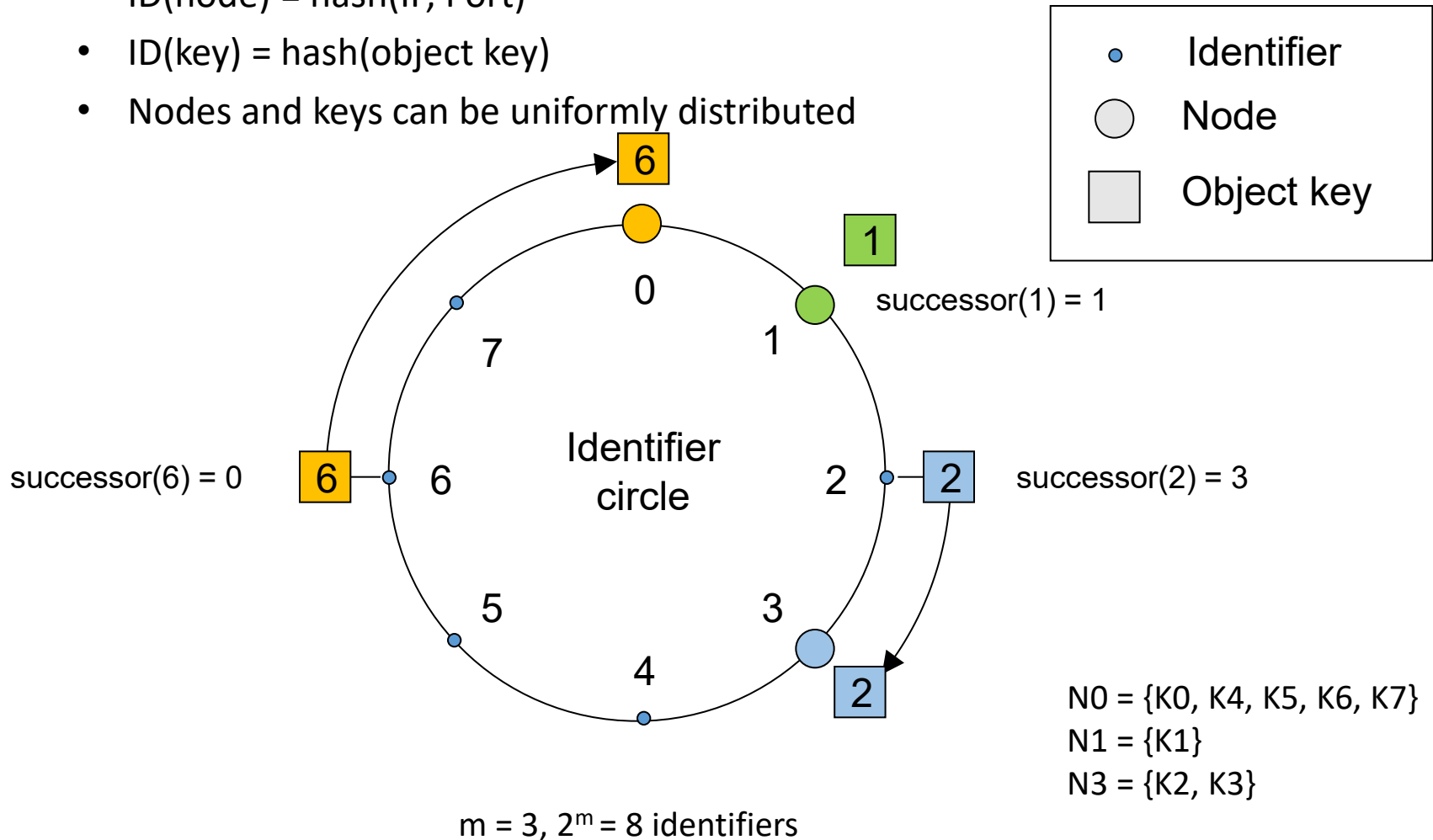
$N_0 = \{K_0, K_4, K_5, K_6, K_7\}$

$N_1 = \{K_1\}$

$N_3 = \{K_2, K_3\}$

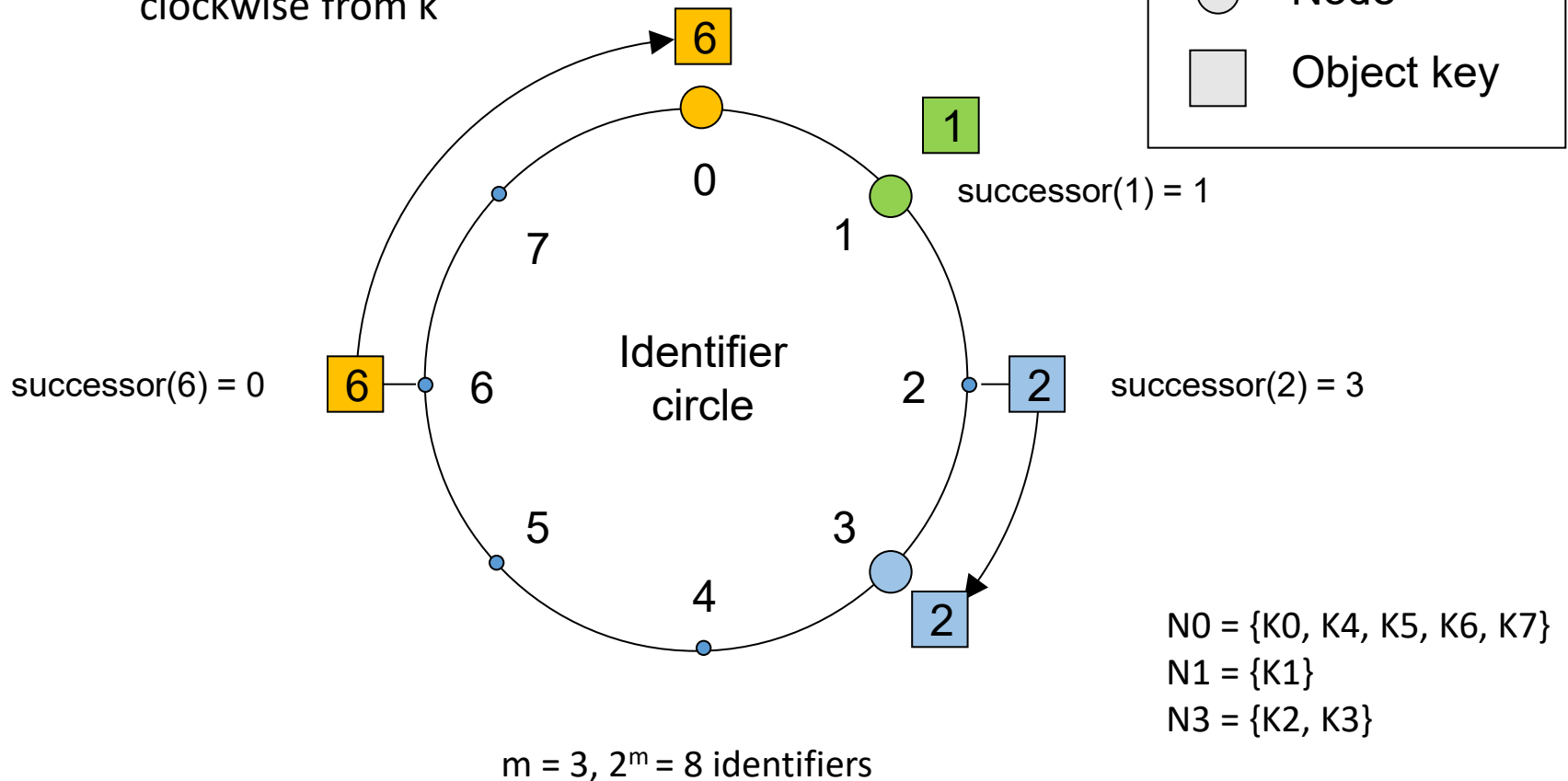
Consistent Hashing

- Both nodes and object keys exist in the same m-bit ID space
- $ID(\text{node}) = \text{hash}(\text{IP, Port})$
- $ID(\text{key}) = \text{hash}(\text{object key})$
- Nodes and keys can be uniformly distributed



Consistent Hashing

- Consistent hashing partitions the key space among nodes
- A key k is assigned to the node whose identifier is equal to or greater than the key's identifier
- This node is called $\text{successor}(k)$ and is the first node clockwise from k



Consistent Hashing

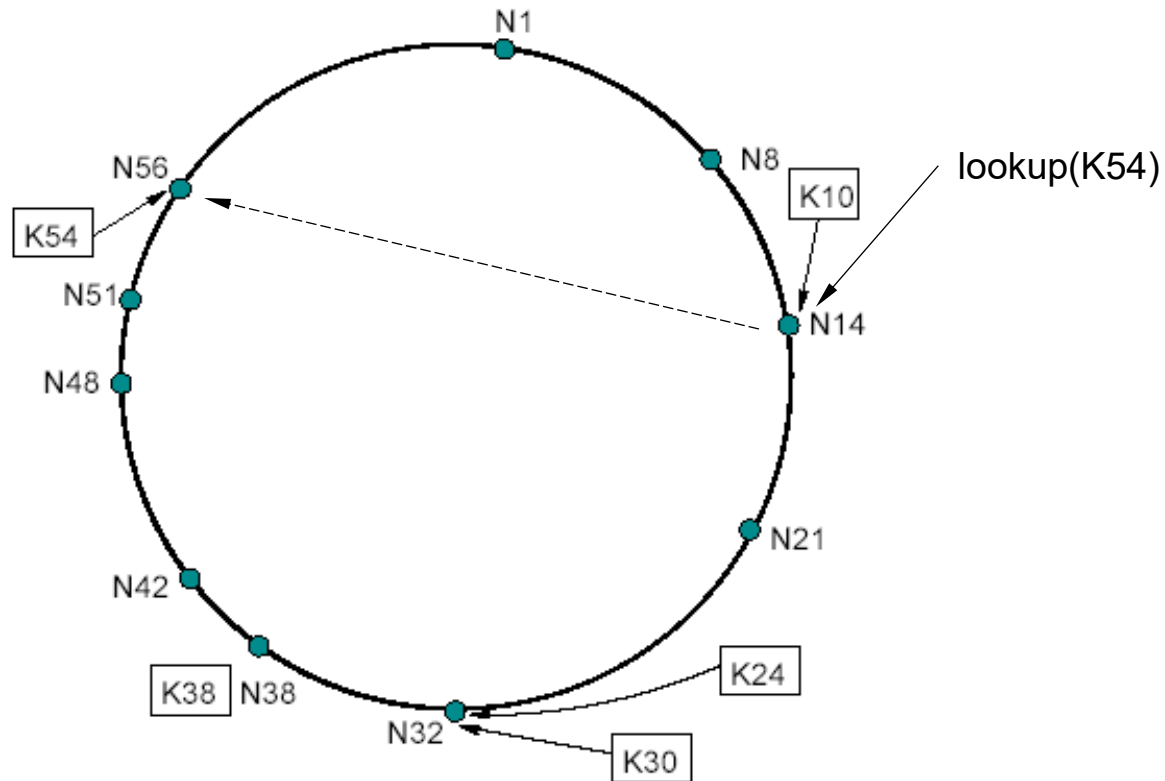
- Assuming the network has N nodes, each node has one hash bucket and the key space is uniformly divided among N nodes
 - A node owns (about) $1/N^{\text{th}}$ of the key space
 - Note that this does not say anything about actual number of keys in the hash bucket of a node
- We are interested in the cost of key lookup, and node join and departure/failure

Key lookup:

- Any node in the network can receive a key lookup request from the outside world.
- The receiving node may not be the owner of the requested key k and the request must be routed to the node that owns the key space that k belongs to.
- Each node maintains a routing table of size $O(N)$: information about other $N-1$ nodes in the network. It's easy to see, key lookup requires $O(1)$ time.

Consistent Hashing

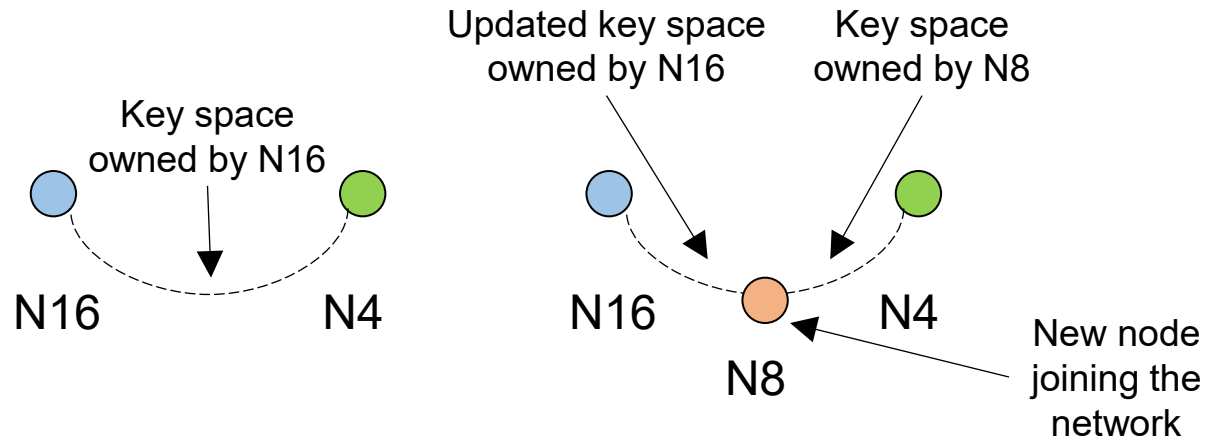
Key lookup:



$m = 6, 2^m = 64$ identifiers

Consistent Hashing

Node Join:



- When a new node joins the network, only one existing node needs to update its hash bucket, move some of its keys to the new node (N16 moves some of the keys from its bucket to N8's bucket) - achieves the smoothness goal.
- All nodes in the network must update their respective routing tables to add information about the newly added node.
- However, now the share of key space owned by each of N16 and N8 is no longer $1/N^{\text{th}}$ - load imbalance is still an issue.

Consistent Hashing

Node departure/failure:

- When a node n departs/fails, a new successor (a node) takes over the key space previously owned by n .
- Only localized update occurs; keys are not moved* - achieves the smoothness goal.
- All nodes in the network must update their respective routing tables.
- However, now the share of the key space owned by new successor is $2/N^{\text{th}}$ - load imbalance is still an issue.

Improving load distribution through introduction of virtual nodes or vnodes:

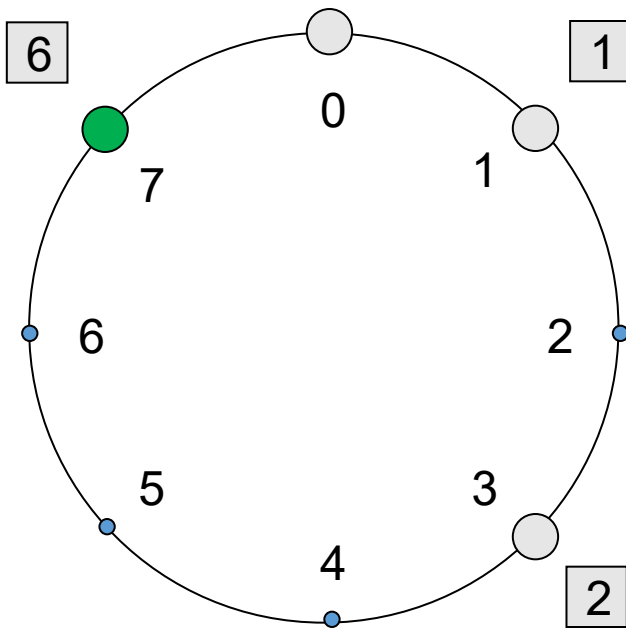
- Each physical node is assigned V random virtual nodes. Therefore, the share of the key space owed by each vnode is $1/(V \times N^{\text{th}})$.

* Assuming the departing node does not move the keys to the new successor and no replication mechanism is in place

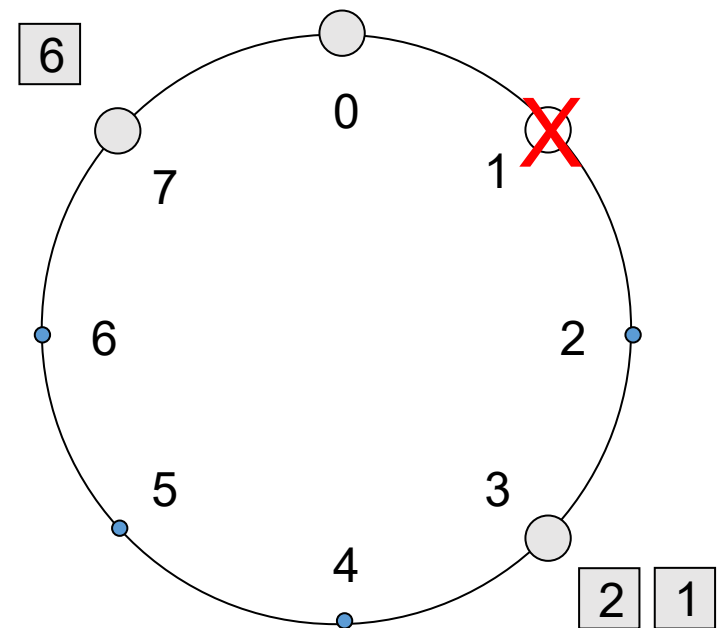
Consistent Hashing: node joins and departures

N7 joins the network

$\text{successor}(6) = 7$



N1 leaves the network



$\text{successor}(1) = 3$

Consistent Hashing

	Consistent Hashing
Size of the routing table on each node	$O(N)$
Key lookup	$O(1)$
Node join	$O(N)$
Node departure/failure	$O(N)$
Key movement as a result of node join or departure/failure	$O(1)$

After node join/departure, to update the routing table, #nodes must be contacted in the worst case

Distributed Hash Table (DHT)

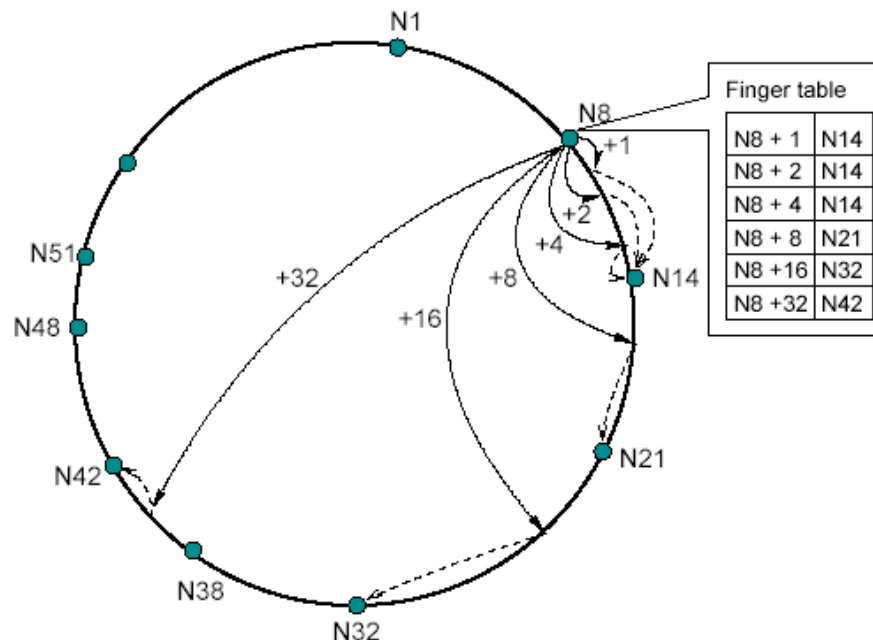
- Distributed Hash Tables inspired by the Consistent Hashing algorithm:
 - Chord
 - Pastry
 - Tapestry
 - CAN
 - Kademlia

- The Chord paper:

Ion Stoica, Robert Morris, David Karger, M. Frans Kaashoek, and Hari Balakrishnan. 2001. Chord: A scalable peer-to-peer lookup service for internet applications. In Proceedings of the 2001 conference on Applications, technologies, architectures, and protocols for computer communications (SIGCOMM '01). Association for Computing Machinery, New York, NY, USA, 149–160.

Chord DHT

- Identifiers are ordered on an identifier circle modulo 2^m , the same as in the Consistent Hashing algorithm
- A new routing protocol for key lookup, adds new links to the overlay network
- The identifier ring is called a Chord ring



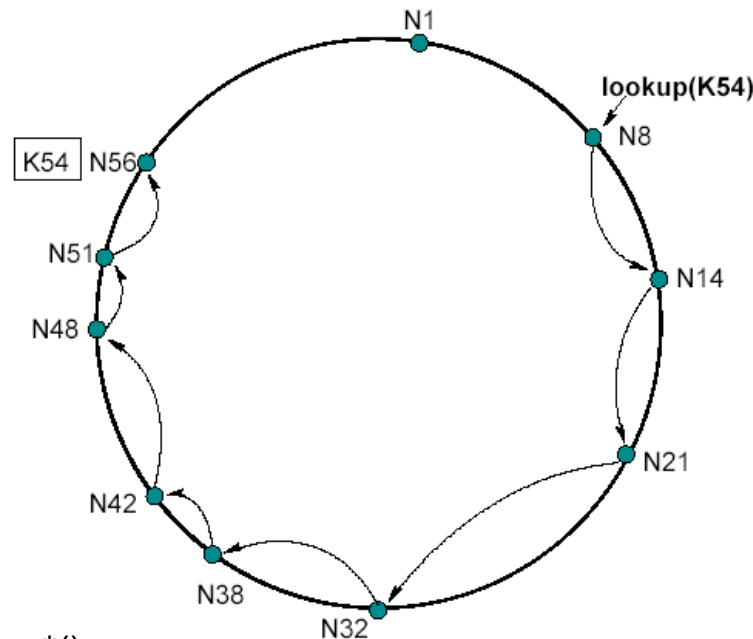
Chord DHT

- In Consistent Hashing, each node maintains a routing table of size $O(N)$ to achieve $O(1)$ key lookup time
- An up to date routing table is required at all times for the system to function correctly
- When N is large, e.g., $10^3 - 10^6$, a routing table of size $O(N)$ can become expensive to maintain
- In practice, in a very large network,
 - It is challenging to ensure every node has up to date information about all other nodes in the network at all times
 - The overhead of updating large routing tables, because of node joins and departures, can lead to reduced system throughput

Chord DHT

Simple approach to key lookup, fixed size routing table:

- Every node n on the ring only knows the identity of the first node clockwise from n (successor* node of n) and first node counterclockwise from n (predessor node of n): only two entries in the routing table of a node.
- Lookup query for key k is passed around the ring until $\text{successor}(k)$ is found. Result is returned along the reverse path: $O(N)$ lookup/routing time and $O(N)$ messages.

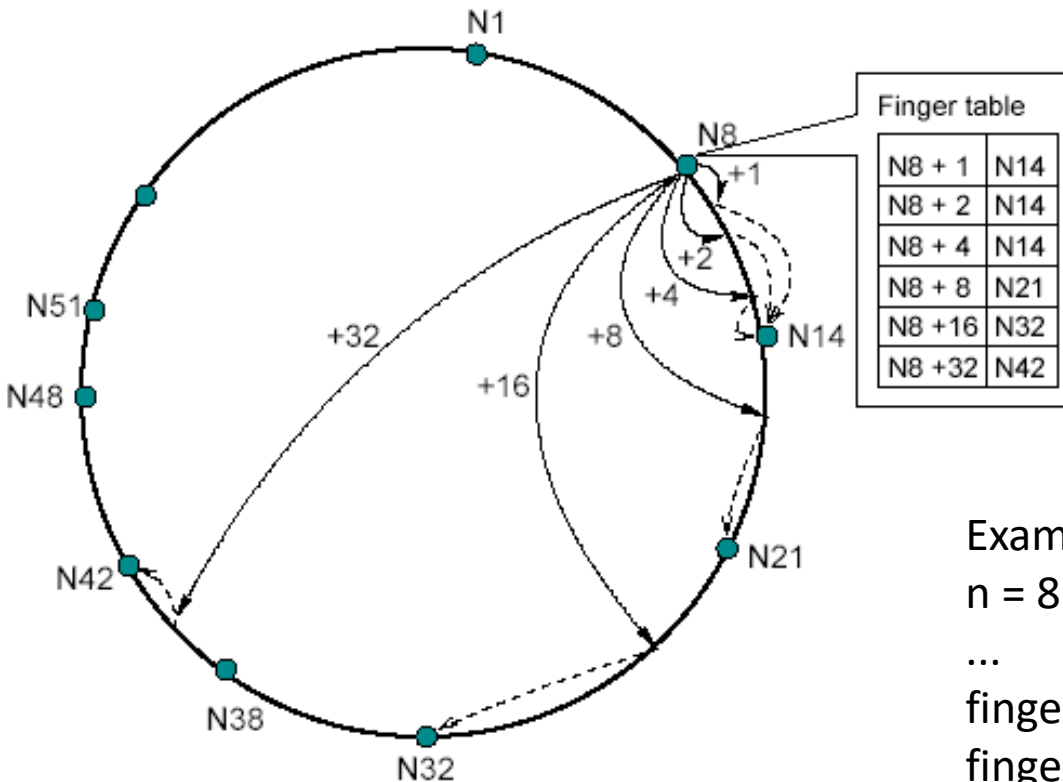


Note: “ $\text{successor}()$ ” and “ successor^* ()” are for keys and nodes, respectively.

Chord DHT

A more efficient solution - create “finger” tables on nodes to speedup routing for key lookup:

- Every node n knows $O(\log N)$ nodes on the ring, i.e., the routing table size is $O(\log N)$



- The i^{th} entry in the table at node n contains the first node s that succeeds n by at least 2^{i-1} where $1 \leq i \leq m$
- $s = \text{successor}((n + 2^{i-1}) \bmod 2^m)$
- s is called the i^{th} finger of node n
- m entries in the finger table of n

Example:

$n = 8$ (N8 in the figure), $m = 6$

...

$\text{finger}[4] = \text{successor}((n + 2^3) \bmod 2^m) = 21$

$\text{finger}[5] = \text{successor}((n + 2^4) \bmod 2^m) = 32$

$\text{finger}[6] = \text{successor}((n + 2^5) \bmod 2^m) = 42$

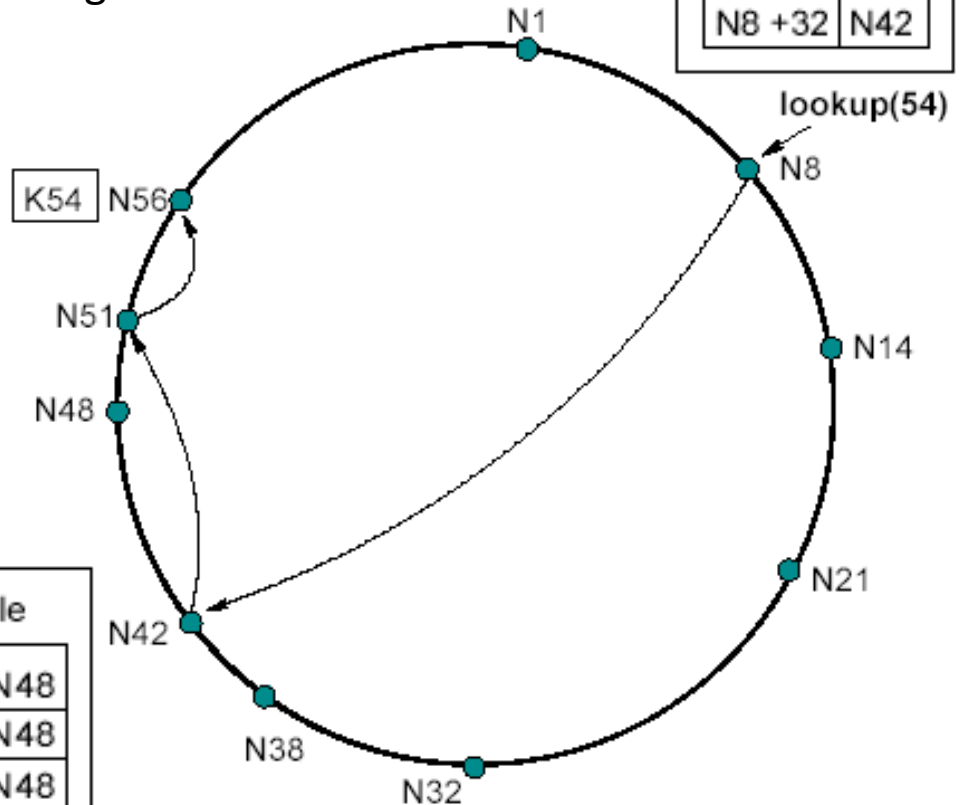
Note: Logarithm to the base 2

Chord DHT

Key lookup using finger table-based routing:

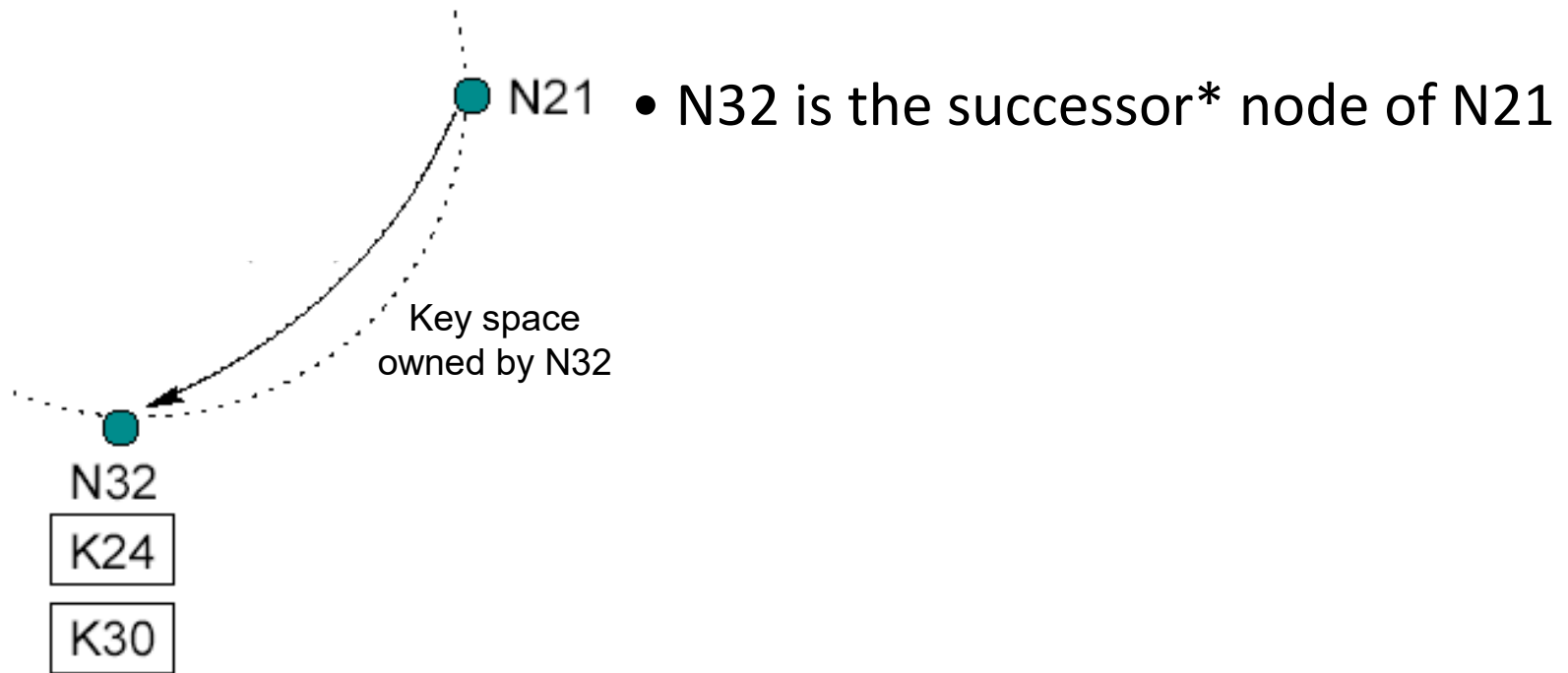
- Node n received a lookup request for the key k
- If k belongs to n 's key space then $\text{successor}(n)$ is returned
- Otherwise, the request is forwarded to a node q
- q is a node in n 's finger table that most immediately precedes k (on the ring)
- Repeat the above two steps until node $\text{successor}(k)$ is found
- At most $O(\log N)$ nodes need to be contacted before reaching $\text{successor}(k)$, the node that has the key k

Finger table	
$N_{42} + 1$	N48
$N_{42} + 2$	N48
$N_{42} + 4$	N48
$N_{42} + 8$	N51
$N_{42} + 16$	N1
$N_{42} + 32$	N14

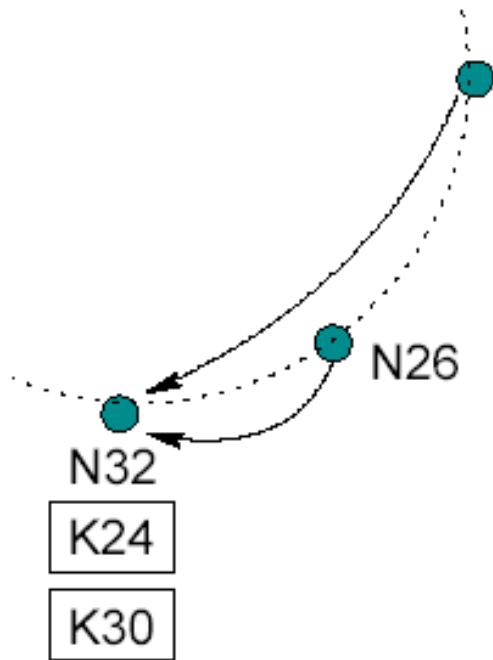


Finger table	
$N_8 + 1$	N14
$N_8 + 2$	N14
$N_8 + 4$	N14
$N_8 + 8$	N21
$N_8 + 16$	N32
$N_8 + 32$	N42

Chord DHT: node join and stabilization

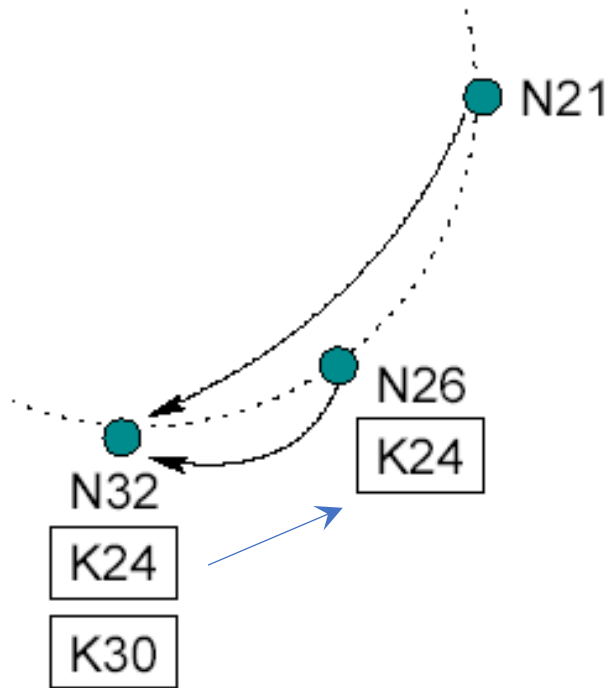


Chord DHT: node join and stabilization



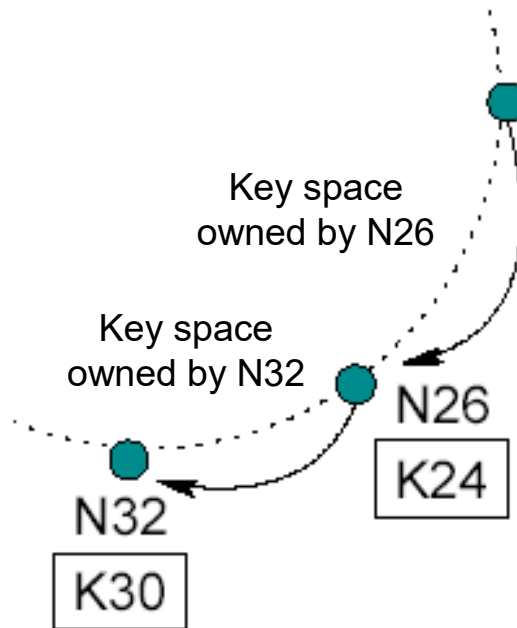
- N26 joins the network
- N26 acquires N32 as its successor* node
- N26 notifies N32
- N32 acquires N26 as its predecessor node

Chord DHT: node join and stabilization



- N32 copies some keys over to K24
- N21 runs the stabilize procedure and asks its successor* node N32 for its predecessor node which is N26

Chord DHT: node join and stabilization



- N21 acquires N26 as its successor* node
- N21 notifies N26 of its existence
- N26 acquires N21 as predecessor node

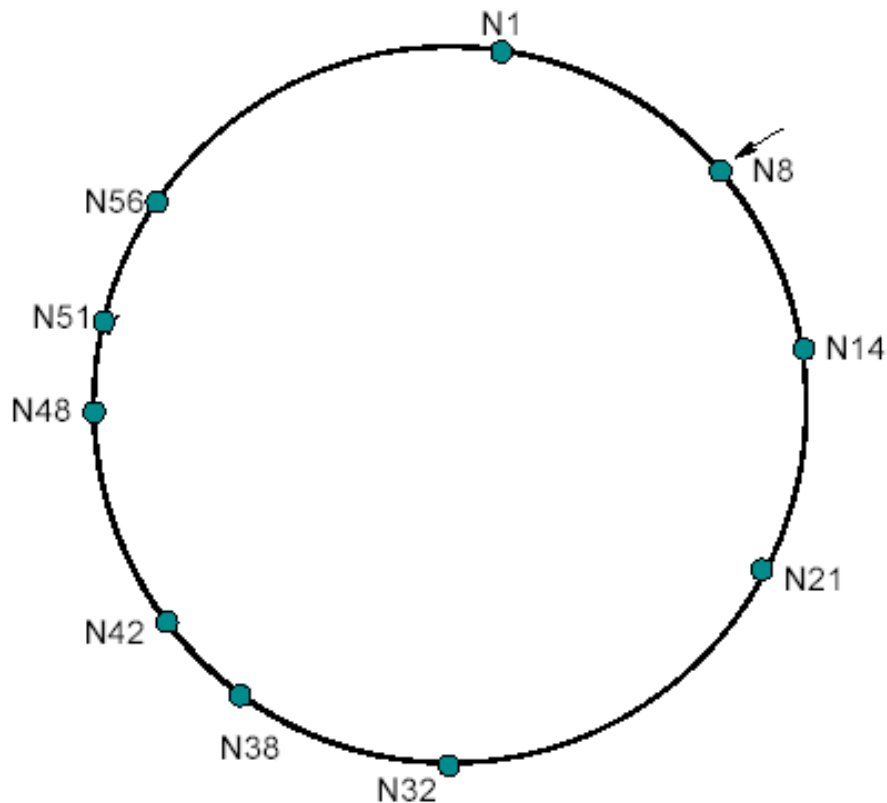
Chord DHT: node join and stabilization

- Correctness relies on nodes maintaining valid successor* pointers
- Each node periodically runs “stabilization” protocol which updates successor* pointers and finger tables; helps existing nodes to learn about newly joined nodes
- What happens if predecessor information is not maintained?

Chord DHT: node departure/failure

- In the even of node failures or departures, successor* pointers and finger tables must be updated
- Voluntary node departures:
 - A node that is about to leave may transfer its keys to its successor* node before it departs

Chord DHT: node departure/failure

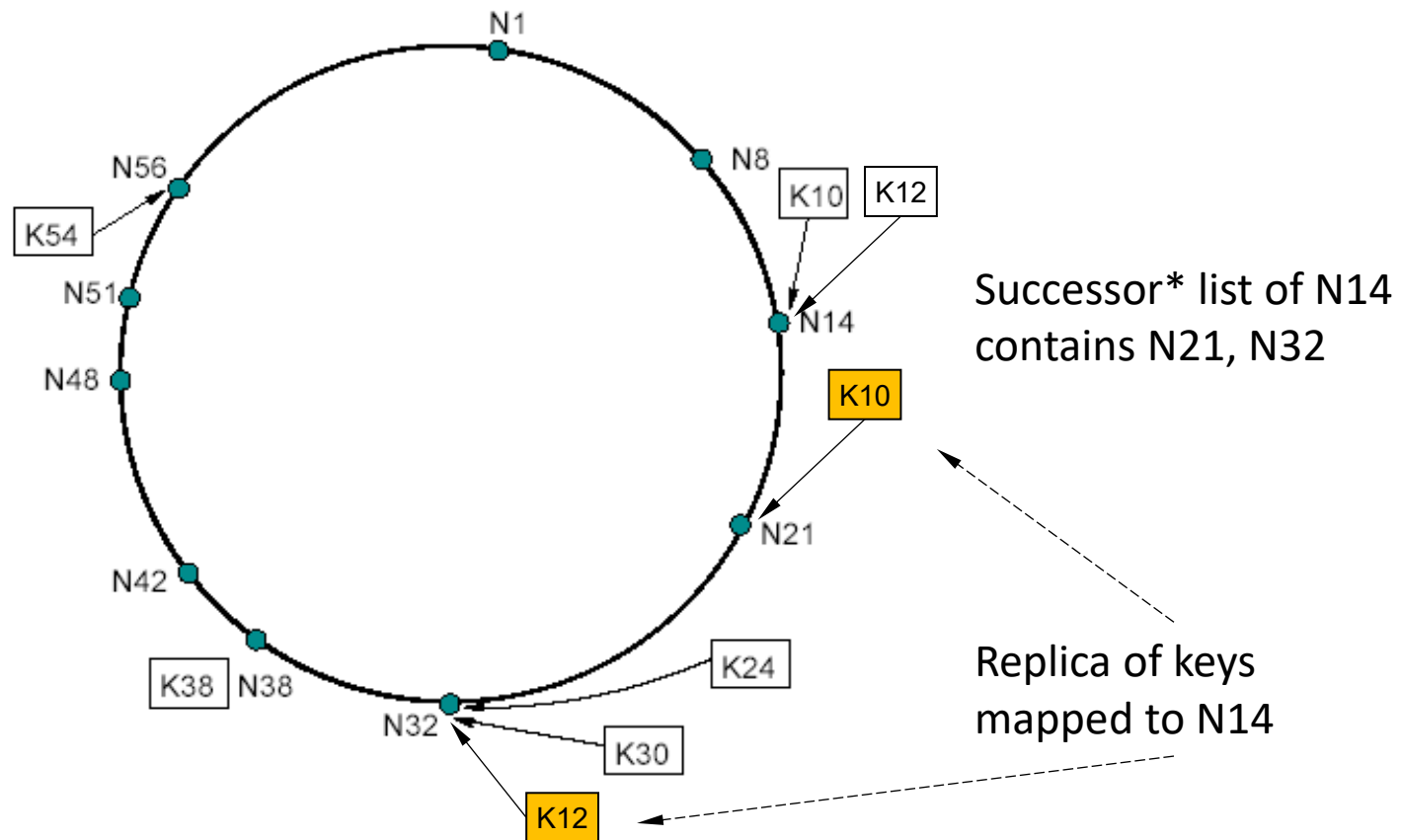


Finger table	
N8 + 1	N14
N8 + 2	N14
N8 + 4	N14
N8 + 8	N21
N8 + 16	N32
N8 + 32	N42

- What happens, if N14, N21, N32 fail simultaneously?
- How N8 would update its finger table after the failure?

Chord DHT: fault tolerance

- Chord must ensure that each node's successor* pointer is up to date
- Each node maintains a successor* list of R nearest successor* nodes
- Replicate objects (key-value pairs) in R successor* nodes



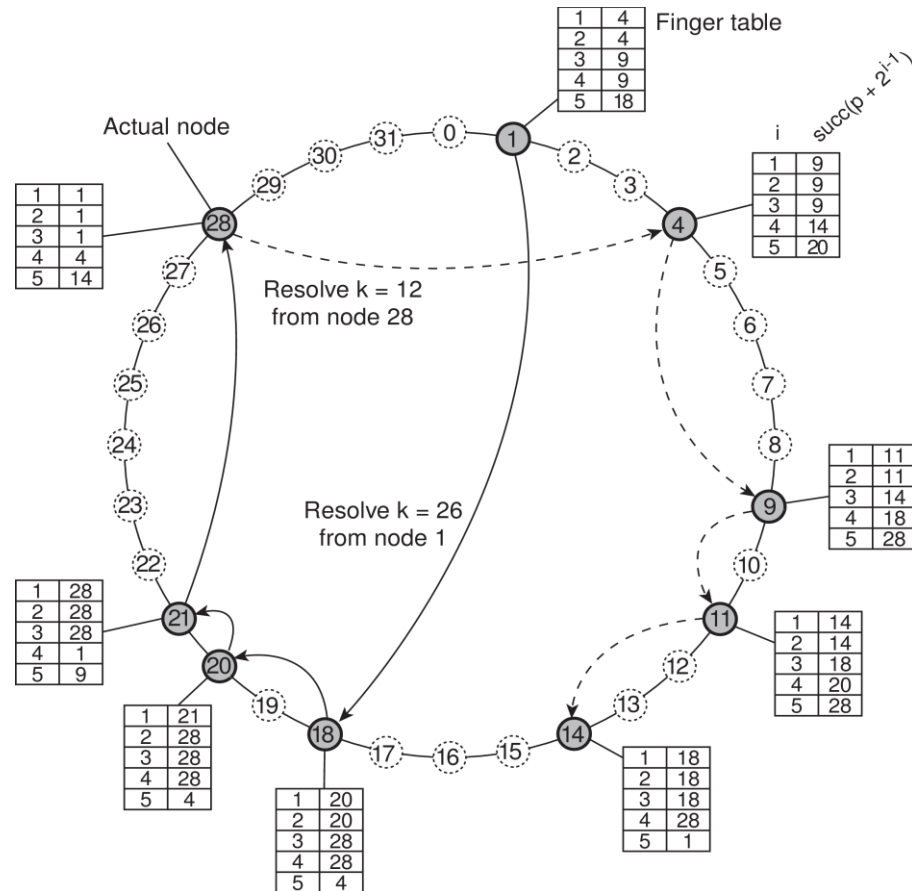
Consistent Hashing vs. Chord DHT

	Consistent Hashing	Chord DHT
Size of the routing table on each node	$O(N)$	$O(\log N)$
Key lookup	$O(1)$	$O(\log N)$
Node join	$O(N)$	$O(\log N)$
Node departure/failure	$O(N)$	$O(\log N)$
Key movement as a result of node join or departure/failure	$O(1)$	$O(1)$

After node join/departure, to update the routing table, #nodes must be contacted in the worst case

Chord DHT

- See section 6.2.3 in the textbook
- Example: resolving key 26 from node 1 and key 12 from node 28



References

- David Karger, Eric Lehman, Tom Leighton, Rina Panigrahy, Matthew Levine, and Daniel Lewin. 1997. Consistent hashing and random trees: distributed caching protocols for relieving hot spots on the World Wide Web. In Proceedings of the twenty-ninth annual ACM symposium on Theory of computing (STOC '97). Association for Computing Machinery, New York, NY, USA, 654–663. <https://doi.org/10.1145/258533.258660>
- Ion Stoica, Robert Morris, David Karger, M. Frans Kaashoek, and Hari Balakrishnan. 2001. Chord: A scalable peer-to-peer lookup service for internet applications. In Proceedings of the 2001 conference on Applications, technologies, architectures, and protocols for computer communications (SIGCOMM '01). Association for Computing Machinery, New York, NY, USA, 149–160. <https://doi.org/10.1145/383059.383071>
- Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Voss, and Werner Vogels. 2007. Dynamo: amazon's highly available key-value store. In Proceedings of twenty-first ACM SIGOPS symposium on Operating systems principles (SOSP '07). Association for Computing Machinery, New York, NY, USA, 205–220. <https://doi.org/10.1145/1294261.1294281>
- Erik Nygren, Ramesh K. Sitaraman, and Jennifer Sun. 2010. The Akamai network: a platform for high-performance internet applications. SIGOPS Oper. Syst. Rev. 44, 3 (July 2010), 2–19. <https://doi.org/10.1145/1842733.1842736>
- M. Van Steen and A. S. Tanenbaum, Distributed Systems, 4th Edition, Pearson, 2023.
- Chord presentation slides at https://resources.mpi-inf.mpg.de/departments/d5/teaching/ws03_04/p2p-data/11-18-paper1.ppt
- Chord presentation slides from University Of Patras.
- Apache Cassandra: <https://cassandra.apache.org/doc/latest/cassandra/architecture/dynamo.html>